TREE BASICS

A tree = connected graph with n nodes, n-1 edges, no cycles. Use adjacency list.

```
vector<vector<int>> adj(n);  // same structure as a graph
vector<int> depth(n, 0), subtreeSize(n, 1);
void dfsTree(int u, int parent) {
    for (int v : adj[u]) {
        if (v == parent) continue;  // skip edge back to parent
        depth[v] = depth[u] + 1;
        dfsTree(v, u);
        subtreeSize[u] += subtreeSize[v];
    }
}
// call dfsTree(root, -1) — root has no real parent
// leaf node: adj[u].size()==1 && u != root  // diameter: 2x BFS/DFS from any node
```

GRID-BASED BFS / DFS

Use when: pathfinding or region exploration on a 2D grid — treat cells as graph nodes.

```
int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1};  // 4-directional moves
bool inBounds(int r,int c,int R,int C) { return r>=0&&r<R&&c>=0&&c<C; }
```

GRID BFS — shortest path in unweighted grid

```
vector<vector<int>> dist(R, vector<int>(C, -1));
queue<pair<int,int>> q;
dist[sr][sc] = 0; q.push({sr, sc});
while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d=0;d<4;d++) {
        int nr=r+dr[d], nc=c+dc[d];
        if (inBounds(nr,nc,R,C) && dist[nr][nc]==-1 && grid[nr][nc]!='#') {
            dist[nr][nc] = dist[r][c] + 1;
            q.push({nr, nc});
        }
    }
}
```

GRID DFS — mark/explore a connected region

```
vector<vector<bool>> visited(R, vector<bool>(C, false));
void dfsGrid(int r, int c) {
    if (!inBounds(r,c,R,C) || visited[r][c] || grid[r][c]=='#') return;
    visited[r][c] = true;
    for (int d=0;d<4;d++) dfsGrid(r+dr[d], c+dc[d]);
}
```

===== READ GRID INPUT (common format) =====

```
vector<string> grid(R);
for (int i=0;i<R;i++) cin >> grid[i];   // grid[r][c] access, chars not ints
```

===== 8-DIRECTIONAL MOVEMENT (diagonals included) =====

```
int dr8[] = {-1,-1,-1,0,0,1,1,1};
int dc8[] = {-1,0,1,-1,1,-1,0,1};
```

===== START DFS/BFS FROM EVERY CELL =====

```
for(int r=0;r<R;r++)
for(int c=0;c<C;c++)
    if(!visited[r][c]) dfsGrid(r,c);
```